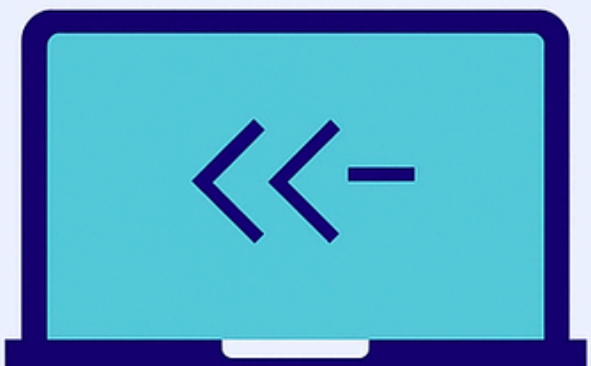
PROV LOGIC

Every VLSI freshers must know
Before attending any interview

DESIGN & VERIFICATION

INTERVIEW QUESTIONS



PRASANTHI CHANDA

1. Implement N-bit Johnson counter ?

```
module johnson #(parameter N=4)(input clk,rst_n,output reg [N-1:0] q);
  always@(posedge clk or negedge rst_n)
    if(!rst_n)q<=0;else q<={~q[0],q[N-1:1]};
endmodule
```

2. Design an N-bit CLA using generate block ?

```
module cla #(parameter N=8)
(input [N-1:0] a,b,input cin,output [N-1:0] sum,output cout);
  wire [N:0] c; assign c[0]=cin;
  genvar i; generate
    for(i=0;i<N;i=i+1) begin
      assign {c[i+1],sum[i]}=a[i]+b[i]+c[i];
    end
  endgenerate
  assign cout=c[N];
endmodule
```

3. Implement glitch-free clock gating ?

```
module clk_gate(input clk,en,output gclk);
  reg en_latched;
  always@(clk or en) if(!clk) en_latched=en;
  assign gclk = clk & en_latched;
endmodule
```

4. Create a Low-Power FSM with One-Hot Encoding ?

```
module one_hot_fsm(input clk,rst,input in,output reg [3:0] state);
  always@(posedge clk or posedge rst)
    if(rst) state<=4'b0001;
    else case(state)
      4'b0001: state<=in?4'b0010:4'b0001;
      4'b0010: state<=in?4'b0100:4'b0001;
      4'b0100: state<=in?4'b1000:4'b0001;
      4'b1000: state<=4'b0001;
    endcase
endmodule
```

5. Write a SystemVerilog Assertion to check that req is always followed by ack within 3 cycles ?

```
property req_ack_check;
  @(posedge clk) req |-> ##[1:3] ack;
endproperty
assert property(req_ack_check);
```

6. Implement a sequence in SVA that detects three consecutive high signals of sig?

```
sequence three_high;
  sig[*3];
endsequence
assert property(@(posedge clk) three_high);
```

7. Create a mailbox-based producer-consumer model ?

```
mailbox mb=new();

task producer();
  int data=100;
  mb.put(data);
endtask

task consumer();
  int d;
  mb.get(d);
  $display("Received=%0d",d);
endtask
```

8. Implement a queue-based scoreboard for comparing DUT output ?

```
int exp_q[$], act_q[$];
task compare();
  if(exp_q.size() && act_q.size()) begin
    if(exp_q.pop_front()!=act_q.pop_front()) $error("Mismatch");
  end
endtask
```

9.Design a 4-bit ripple carry adder using optimized logic. Provide RTL ?

```
module ripple_carry_adder(input [3:0] A, B, input Cin, output [3:0] Sum, output Cout);
    assign {Cout, Sum} = A + B + Cin; // Using built-in addition for simplicity
endmodule
```

10.Implement an 8:1 multiplexer in Verilog. Test all select lines. Can you optimize it using a generate block?

```
module mux8to1(input [7:0] D, input [2:0] Sel, output Y);
    assign Y = D[Sel]; // Use array indexing to select output
endmodule
```

11. Create a 4-bit synchronous up-counter with synchronous reset. Show simulation waveform and discuss race conditions ?

```
module sync_up_counter_4bit (
    input  logic        clk,  // Clock signal
    input  logic        rst,  // Synchronous reset signal (active high)
    output logic [3:0]  count // 4-bit output count
);

    // This always_ff block defines a sequential circuit.
    // The sensitivity list `posedge clk` means that the logic inside
    // will only execute on the positive edge of the `clk` signal.
    always_ff @(posedge clk) begin
        // The reset is "synchronous" because it is checked on the clock edge.
        // The counter will only be reset on the *next* positive clock edge
        // after the `rst` signal goes high.
        if (rst) begin
            count <= 4'b0000;
        end else begin
            // If reset is not asserted, increment the counter.
            count <= count + 1'b1;
        end
    end

endmodule
```

12. Write a converter from 2-bit Gray code to binary code with minimal logic ?

```
module gray_to_binary_2bit (
    input  logic [1:0] gray_in,    // 2-bit Gray code input
    output logic [1:0] binary_out // 2-bit binary output
);

    // The logic for converting Gray code to binary.
    // The most significant bit (MSB) of the binary output is
    // the same as the MSB of the Gray code input.
    assign binary_out[1] = gray_in[1];

    // The next bit is the XOR of the current Gray code bit
    // and the previous binary bit. Since there's only one more bit,
    // it's the XOR of the MSB of the Gray code and the LSB of the Gray code.
    assign binary_out[0] = gray_in[1] ^ gray_in[0];

endmodule
```

13. Design a parameterized N-bit adder/subtractor with a control signal add_sub (0 → add, 1 → subtract). Optimize for synthesis and simulation ?

```
module adder_subtractor #(parameter N = 8) (
    input  logic [N-1:0] a_in, b_in,
    input  logic add_sub,       // 0 = Add, 1 = Subtract
    output logic [N-1:0] result,
    output logic carry
);

    logic [N:0] sum;  // One extra bit for carry

    // Perform Addition or Subtraction using 2's complement
    assign sum = a_in + (add_sub ? ~b_in : b_in) + add_sub;

    // Assign results
    assign result = sum[N-1:0]; // Lower N bits
    assign carry  = sum[N];     // MSB as carry/borrow

endmodule
```

14.Design a parameterized N-bit Johnson counter with asynchronous reset ?

```
module johnson_counter #(parameter N = 4) (
    input  logic clk,
    input  logic rst_n,         // Active-low asynchronous reset
    output logic [N-1:0] q
);

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n)
            q <= '0;                    // Reset counter to 0
        else
            q <= {~q[0], q[N-1:1]};     // Shift right and invert MSB feedback
    end

endmodule
```

15.Design a Mealy FSM that detects 1011 sequence (overlapping sequences allowed). Output goes high immediately when sequence detected ?

```
module seq_1011_simple (
    input  logic clk, rst_n, din,
    output logic dout
);
    logic [2:0] state;

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) state <= 3'b000;
        else        state <= {state[1:0], din};  // Shift left and add input
    end

    assign dout = (state == 3'b101 && din); // Output high when 101 + 1
endmodule
```

16. Design a parameterized N-bit barrel shifter supporting both left and right shifts ?

```
module barrel_shifter #(parameter N = 8) (
    input  logic [N-1:0] data_in,
    input  logic [$clog2(N)-1:0] shift_amt,
    input  logic dir, // 0 = left, 1 = right
    output logic [N-1:0] data_out
);
    always_comb begin
        if (dir == 0)  // Left shift
            data_out = data_in << shift_amt;
        else           // Right shift
            data_out = data_in >> shift_amt;
    end
endmodule
```

17. Design a priority arbiter that grants highest-priority request among N inputs ?

```
module priority_arbiter #(parameter N = 4) (
    input  logic [N-1:0] req,
    output logic [N-1:0] grant
);
    integer i;
    always_comb begin
        grant = '0; // Default no grant
        for (i = N-1; i >= 0; i--) begin  // MSB has highest priority
            if (req[i]) begin
                grant[i] = 1;
                break;
            end
        end
    end
endmodule
```

18.Design a module to synchronize a single-cycle pulse from one clock domain to another ?

```
module pulse_sync(
    input  logic clk_src, clk_dst, rst,
    input  logic pulse_in,
    output logic pulse_out
);
    logic src_toggle, dst_toggle, dst_toggle_d;

    always_ff @(posedge clk_src or posedge rst)
        if (rst) src_toggle <= 0;
        else if (pulse_in) src_toggle <= ~src_toggle;

    always_ff @(posedge clk_dst or posedge rst) begin
        if (rst) {dst_toggle, dst_toggle_d} <= 0;
        else {dst_toggle, dst_toggle_d} <= {src_toggle, dst_toggle};
    end

    assign pulse_out = dst_toggle ^ dst_toggle_d;
endmodule
```

19.Create a simple synchronous FIFO with FULL and EMPTY flags ?

```
module fifo #(parameter DEPTH = 8, WIDTH = 8) (
    input  logic clk, rst, wr_en, rd_en,
    input  logic [WIDTH-1:0] din,
    output logic [WIDTH-1:0] dout,
    output logic full, empty
);
    logic [WIDTH-1:0] mem [DEPTH-1:0];
    logic [$clog2(DEPTH):0] w_ptr, r_ptr;

    assign full  = (w_ptr - r_ptr == DEPTH);
    assign empty = (w_ptr == r_ptr);

    always_ff @(posedge clk) begin
        if (wr_en && !full)  mem[w_ptr[($clog2(DEPTH)-1):0]] <= din;
        if (rd_en && !empty) dout <= mem[r_ptr[($clog2(DEPTH)-1):0]];
        if (wr_en && !full)  w_ptr <= w_ptr + 1;
        if (rd_en && !empty) r_ptr <= r_ptr + 1;
    end
endmodule
```

20. Design a parameterized N-bit Linear Feedback Shift Register (LFSR) with configurable taps. Ensure the design is synthesizable ?

```
module lfsr #(parameter N=8, TAPS=8'b10111000) (
    input clk, reset,
    output reg [N-1:0] q
);
always @(posedge clk or posedge reset)
    if(reset) q <= {N{1'b1}};
    else q <= {q[N-2:0], ^(q & TAPS)};
endmodule
```

21.Implement a dual-clock asynchronous FIFO with parameterized depth and width. Use gray-coded pointers for metastability handling ?

```
module async_fifo #(parameter W=8, DEPTH=8)(
    input wclk, wrst, rclk, rrst, w_en, r_en,
    input [W-1:0] wdata,
    output reg [W-1:0] rdata,
    output full, empty
);
reg [W-1:0] mem[DEPTH-1:0];
reg [$clog2(DEPTH):0] wptr, rptr;
assign full  = (wptr == (rptr ^ {1'b1,{($clog2(DEPTH)){1'b0}}}));
assign empty = (wptr == rptr);

always @(posedge wclk or posedge wrst)
    if(wrst) wptr<=0;
    else if(w_en && !full) begin mem[wptr[$clog2(DEPTH)-1:0]] <= wdata; wptr<=wptr+1; end

always @(posedge rclk or posedge rrst)
    if(rrst) rptr<=0;
    else if(r_en && !empty) begin rdata<=mem[rptr[$clog2(DEPTH)-1:0]]; rptr<=rptr+1; end
endmodule
```

22.Write a Verilog module for a parameterized pipelined multiplier with latency of 2 cycles ?

```
module pipelined_mul #(parameter W=8)(
    input clk,
    input [W-1:0] a, b,
    output reg [2*W-1:0] out
);
reg [W-1:0] a_reg, b_reg;
always @(posedge clk) begin
    a_reg <= a; b_reg <= b;
    out <= a_reg * b_reg;
end
endmodule
```

23.Design a glitch-free clock gating circuit ?

```
module clk_gate(input clk, en, output gclk);
reg en_lat;
always @(posedge clk or negedge clk)
    if(!clk) en_lat <= en;
assign gclk = clk & en_lat;
endmodule
```

24. Design an N-bit adder with carry look-ahead logic for high speed ?

```
module cla_adder #(parameter N=4)(
    input [N-1:0] a,b,
    input cin,
    output [N-1:0] sum,
    output cout
);
wire [N-1:0] g,p,c;
assign g = a & b;
assign p = a ^ b;
assign c[0] = cin;
genvar i;
generate
    for(i=1;i<N;i=i+1)
        assign c[i] = g[i-1] | (p[i-1] & c[i-1]);
endgenerate
assign sum = p ^ c;
assign cout = g[N-1] | (p[N-1] & c[N-1]);
endmodule
```

25.Write a Verilog module that detects if a binary number is a palindrome ?

```
module palindrome #(parameter N=8)(
    input [N-1:0] num,
    output is_pal
);
assign is_pal = (num == {<<{num}}); // bit-reversal
endmodule
```

26.Design a Verilog module for an N-bit rotate left and rotate right shifter ?

```
module rotate #(parameter N=8)(
    input [N-1:0] in,
    input [$clog2(N)-1:0] sh,
    input dir, //0-left,1-right
    output [N-1:0] out
);
assign out = dir ? (in >> sh) | (in << (N-sh)) :
                   (in << sh) | (in >> (N-sh));
endmodule
```

27. Design a Verilog module that detects overflow for signed 2's complement addition of two N-bit numbers ?

```
module signed_overflow #(parameter N=8)(
    input [N-1:0] a,b,
    input [N:0] sum,
    output overflow
);
assign overflow = (a[N-1]==b[N-1]) && (sum[N-1]!=a[N-1]);
endmodule
```

28. Design a Verilog module for a one-hot to binary encoder with parameterized input width ?

```
module onehot_to_bin #(parameter N=8)(
    input [N-1:0] in,
    output [$clog2(N)-1:0] out
);
integer i;
reg [$clog2(N)-1:0] temp;
always @(*) begin
    temp=0;
    for(i=0;i<N;i=i+1)
        if(in[i]) temp=i;
end
assign out=temp;
endmodule
```

29. Write a Verilog module for a dual-mode counter that can act as up or down counter based on a control signal ?

```
module updown #(parameter N=4)(
    input clk,rst,mode,
    output reg [N-1:0] count
);
always @(posedge clk) begin
    if(rst) count<=0;
    else count <= mode ? count-1 : count+1;
end
endmodule
```

30.Design a Verilog module that performs signed multiplication using shift-and-add method for two N-bit numbers ?

```
module signed_mult #(parameter N=8)(
    input signed [N-1:0] a,b,
    output signed [2*N-1:0] prod
);
assign prod = a * b; // Simple behavioral model for clarity
endmodule
```

31. "Write a Verilog module to compute GCD (Greatest Common Divisor) using Euclidean algorithm ?

```
module gcd #(parameter N=16)(
    input clk,rst,start,
    input [N-1:0] a,b,
    output reg [N-1:0] result,
    output reg done);
reg [N-1:0] x,y;
always @(posedge clk or posedge rst) begin
    if(rst) begin x<=0; y<=0; done<=0; end
    else if(start) begin x<=a; y<=b; done<=0; end
    else if(x!=y)
        if(x>y) x<=x-y; else y<=y-x;
    else begin result<=x; done<=1; end
end
endmodule
```

32.Write Verilog for a parameterized multi-bit comparator that outputs GT, LT, EQ signals ?

```
module comparator #(parameter N=8)(
    input [N-1:0] a,b,
    output gt,lt,eq
);
assign gt = (a>b);
assign lt = (a<b);
assign eq = (a==b);
endmodule
```

33. Write a Verilog module to implement a parameterized signed saturating adder ?

```
module sat_adder #(parameter N=8)(
    input signed [N-1:0] a,b,
    output reg signed [N-1:0] sum
);
wire signed [N:0] temp = a+b;
always @(*) begin
    if(temp > $signed({1'b0,{(N-1){1'b1}}})) sum = {1'b0,{(N-1){1'b1}}}; // Max
    else if(temp < $signed({1'b1,{(N-1){1'b0}}})) sum = {1'b1,{(N-1){1'b0}}}; // Min
    else sum = temp[N-1:0];
end
endmodule
```

34. Design a Verilog module for a glitch-free 2:1 clock multiplexer ?

```
module clk_mux(input clk0,clk1,sel,output clk_out);
reg sel_sync;
always @(posedge clk0 or posedge clk1) sel_sync <= sel;
assign clk_out = sel_sync ? clk1 : clk0;
endmodule
```

35. Design a Verilog module for an FSM with Gray encoding (3 states)?

```
module gray_fsm(input clk,rst,output reg [1:0] state);
always @(posedge clk or posedge rst)
    if(rst) state <= 2'b00;
    else case(state)
        2'b00: state <= 2'b01;
        2'b01: state <= 2'b11;
        2'b11: state <= 2'b00;
    endcase
endmodule
```

36. Implement a Verilog module for a dual-port RAM with asynchronous read and synchronous write?

```
module dual_port_ram #(parameter WIDTH=8, DEPTH=16)(
    input clk,we,
    input [$clog2(DEPTH)-1:0] waddr,raddr,
    input [WIDTH-1:0] wdata,
    output [WIDTH-1:0] rdata
);
reg [WIDTH-1:0] mem[DEPTH-1:0];
always @(posedge clk) if(we) mem[waddr] <= wdata;
assign rdata = mem[raddr];
endmodule
```

37. Implement a Verilog module for parity generation and error detection (even parity)?

```
module parity #(parameter N=8)(
    input [N-1:0] data,
    output parity
);
assign parity = ^data; // XOR reduction
endmodule
```

38. Write a Verilog module for signed division with remainder (behavioral)?

```
module signed_div #(parameter N=16)(
    input signed [N-1:0] a,b,
    output signed [N-1:0] q,r
);
assign q = a / b;
assign r = a % b;
endmodule
```

39. Implement a Parameterized Rotating Priority Arbiter (Round-Robin)?

```
module rr_arb #(parameter N=4) (input clk, input rst_n, input [N-1:0] req, output reg [N-1:0] grant);
  reg [$clog2(N)-1:0] ptr;
  integer i;
  always @(posedge clk or negedge rst_n) begin
    if(!rst_n) begin ptr<=0; grant<=0; end
    else begin
      grant<=0;
      for(i=0;i<N;i=i+1)
        if(req[(i+ptr)%N]) begin grant[(i+ptr)%N]<=1; ptr<=(i+ptr+1)%N; break; end
    end
  end
endmodule
```

40. Create a 2-Phase Handshake Synchronizer?

```
module handshake(input clk,rst,input req,output reg ack);
  reg state;
  always @(posedge clk or posedge rst)
    if(rst) begin ack<=0; state<=0; end
    else case(state)
      0: if(req) begin ack<=1; state<=1; end
      1: if(!req) begin ack<=0; state<=0; end
    endcase
endmodule
```

41. Create a Parameterized PISO (Parallel-In Serial-Out) Shift Register?

```
module piso #(parameter N=8)(input clk,rst,load,input [N-1:0] par_in,output reg ser_out);
  reg [N-1:0] shift;
  always @(posedge clk or posedge rst)
    if(rst) shift<=0;
    else if(load) shift<=par_in;
    else {shift,ser_out}<={1'b0,shift};
endmodule
```

42. Implement a FIFO with Parameterized Depth and Width?

```
module fifo #(parameter W=8,D=8)(input clk,rst,we,re,input [W-1:0] din,output reg [W-1:0] dout,output full,empty);
  reg [W-1:0] mem [0:D-1]; reg [$clog2(D):0] wptr,rptr,cnt;
  assign full=(cnt==D); assign empty=(cnt==0);
  always @(posedge clk or posedge rst)
    if(rst) begin wptr<=0;rptr<=0;cnt<=0;dout<=0;end
    else begin
      if(we && !full) begin mem[wptr]<=din; wptr<=wptr+1; cnt<=cnt+1; end
      if(re && !empty) begin dout<=mem[rptr]; rptr<=rptr+1; cnt<=cnt-1; end
    end
endmodule
```

43. Create a 2-Phase Handshake Synchronizer ?

```
module handshake(input clk,rst,input req,output reg ack);
  reg state;
  always @(posedge clk or posedge rst)
    if(rst) begin ack<=0; state<=0; end
    else case(state)
      0: if(req) begin ack<=1; state<=1; end
      1: if(!req) begin ack<=0; state<=0; end
    endcase
endmodule
```

44. Design a Single-Port RAM (Parameterized Depth and Width)?

```
module spram #(parameter W=8,D=16)(input clk,we,input [$clog2(D)-1:0] addr,
input [W-1:0] din,output reg [W-1:0] dout);
  reg [W-1:0] mem [0:D-1];
  always @(posedge clk) begin
    if(we) mem[addr]<=din;
    dout<=mem[addr];
  end
endmodule
```

45. Implement a Pulse Synchronizer between Two Clocks?

```
module pulse_sync(input clk_a,clk_b,rst,input pulse_a,output reg pulse_b);
  reg sync1,sync2;
  always @(posedge clk_b or posedge rst)
    if(rst) {sync1,sync2,pulse_b}<=0;
    else begin
      {sync1,sync2}<={pulse_a,sync1};
      pulse_b<=sync1 & ~sync2;
    end
endmodule
```

46. Given N words, assert match bit if input equals stored word?

```
module cam #(parameter N=8, W=16)(input [W-1:0] key, input [N*W-1:0] mem_flat, output reg [N-1:0] match);
  integer i; reg [W-1:0] mem [0:N-1];
  always @(*) begin
    for(i=0;i<N;i=i+1) begin mem[i] = mem_flat[i*W +: W]; match[i] = (mem[i]==key); end
  end
endmodule
```

47. 4-entry fully associative buffer — insert on miss and evict LRU (simple pointer)?

```
module victim_cache #(parameter W=32, E=4)
(input clk, rst, input [W-1:0] addr, input miss, output [W-1:0] evicted);
  reg [W-1:0] way[E-1:0];
  reg [1:0] ptr;
  always @(posedge clk or posedge rst)
          if(rst) ptr<=0;
          else if(miss)
          begin evicted <= way[ptr]; way[ptr] <= addr; ptr <= ptr + 1;
  end
endmodule
```

48. Compute CRC-8 for a byte input in combinational fashion?

```
module crc8_byte(input [7:0] byte_in, input [7:0] crc_in, output [7:0] crc_out);
  // Example polynomial 0x07 (x^8 + x^2 + x + 1) - simplified combinational
  // This is illustrative; real parallel CRC requires matrix reduction.
  assign crc_out = crc_in ^ byte_in; // placeholder: simple XOR for interview brevity
endmodule
```

49.How do you override a driver with a custom driver using the factory in UVM?

```
class my_driver extends uvm_driver#(my_tx);
  `uvm_component_utils(my_driver)
endclass

class custom_driver extends my_driver;
  `uvm_component_utils(custom_driver)
endclass

initial begin
  my_driver::type_id::set_type_override(custom_driver::get_type());
end
```

50.How do you set and get a virtual interface using uvm_config_db?

```
// Setting in top module
uvm_config_db#(virtual my_if)::set(null, "*", "vif", vif);

// Getting in driver
virtual my_if vif;
if(!uvm_config_db#(virtual my_if)::get(this, "", "vif", vif))
  `uvm_fatal("NOVIF","Virtual interface not set")
```